

01_audio_training

September 27, 2026

1 Audio Training

Train a Model with Audio MNIST data and analyse the results

```
[1]: from sdoml_task1.config import PROJECT_DIR, DATA_DIR, N_MFCC
    from sdoml_task1.features import extract_features, decode_audio, build_feature_dataset
    from sdoml_task1.dataset import AudioMNISTFeaturesDataset
    from sdoml_task1.modeling.model import Net

    from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
    from datasets import load_dataset, Audio
    from torch.utils.data import DataLoader
    import torch.nn.functional as F
    import matplotlib.pyplot as plt
    import torch.optim as optim
    from pathlib import Path
    import torch.nn as nn
    import numpy as np
    import pickle
    import torch
```

```
[2]: ds = load_dataset("gilkeyio/AudioMNIST")
    ds = ds.cast_column("audio", Audio(decode=False))

    X_train, y_train = build_feature_dataset(ds["train"])
    X_test, y_test = build_feature_dataset(ds["test"])

    print(X_train.shape, y_train.shape) # ex: (24000, 26) (24000,)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Extract features: 100%| | 24000/24000 [02:10<00:00, 183.62it/s]

Extract features: 100%| | 6000/6000 [00:31<00:00, 188.54it/s]

(24000, 26) (24000,)

```
[3]: train_dataset = AudioMNISTFeaturesDataset(X_train, y_train)
test_dataset = AudioMNISTFeaturesDataset(X_test, y_test)

# Data loaders
loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=True)
```

1.0.1 MODEL DEFINITION

For training we will use a Multilayer Perceptron with an input of 26 neurons, two hidden layers of 64 and 32 neurons and a final output layer of 10 neurons according to the number of classes in the dataset (0-9 digits).

More details: - Optimizer: Adam (lr = 1e-4) - loss function: Cross Entropy Loss

```
[4]: total_loss_train = []
total_accuracy_train = []
total_loss_valid = []
total_accuracy_valid = []

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
net = Net(input_dim=X_train.shape[1]).to(device)
opt = optim.Adam(net.parameters(), lr=1e-4)
loss_fn = nn.CrossEntropyLoss()

epochs = 40

all_confusion_matrices = [] # Confusion matrix part

for epoch in range(epochs):
    net.train()
    epoch_loss_train = 0.0
    correct_train = 0
    total_train = 0

    all_preds = [] # Confusion matrix part (prediction)
    all_labels = [] # Confusion matrix part (real answer)

    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        out = net(xb)
        loss = loss_fn(out, yb)
        loss.backward()
        opt.step()
        epoch_loss_train += loss.item() * xb.size(0)

    preds = out.argmax(1)
```

```

correct_train += (preds == yb).sum().item()
total_train += yb.size(0)

all_preds.extend(preds.cpu().numpy())
all_labels.extend(yb.cpu().numpy())

cm = confusion_matrix(all_labels, all_preds)
all_confusion_matrices.append(cm)

train_loss = epoch_loss_train / total_train
train_accuracy = correct_train / total_train
total_loss_train.append(train_loss)
total_accuracy_train.append(train_accuracy)

net.eval()
epoch_loss_valid = 0.0
correct_valid = 0
total_valid = 0

with torch.no_grad():
    for xb, yb in test_loader:
        xb, yb = xb.to(device), yb.to(device)
        out = net(xb)
        loss = loss_fn(out, yb)

        epoch_loss_valid += loss.item() * xb.size(0)
        preds = out.argmax(1)
        correct_valid += (preds == yb).sum().item()
        total_valid += yb.size(0)

valid_loss = epoch_loss_valid / total_valid
valid_accuracy = correct_valid / total_valid
total_loss_valid.append(valid_loss)
total_accuracy_valid.append(valid_accuracy)

print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_loss:.4f} | Train Acc:
↪ {train_accuracy:.4f} | Val Loss: {valid_loss:.4f} | Val Acc:
↪ {valid_accuracy:.4f}")

```

```

Epoch 1/40 | Train Loss: 10.2072 | Train Acc: 0.1562 | Val Loss: 2.5304 | Val
Acc: 0.2275
Epoch 2/40 | Train Loss: 2.0032 | Train Acc: 0.3280 | Val Loss: 1.5446 | Val
Acc: 0.4355
Epoch 3/40 | Train Loss: 1.2710 | Train Acc: 0.5462 | Val Loss: 1.0792 | Val
Acc: 0.6105
Epoch 4/40 | Train Loss: 0.8991 | Train Acc: 0.6998 | Val Loss: 0.8337 | Val
Acc: 0.7220

```

Epoch 5/40 | Train Loss: 0.6955 | Train Acc: 0.7910 | Val Loss: 0.6865 | Val
 Acc: 0.7963
 Epoch 6/40 | Train Loss: 0.5690 | Train Acc: 0.8399 | Val Loss: 0.5965 | Val
 Acc: 0.8385
 Epoch 7/40 | Train Loss: 0.4840 | Train Acc: 0.8696 | Val Loss: 0.5355 | Val
 Acc: 0.8482
 Epoch 8/40 | Train Loss: 0.4225 | Train Acc: 0.8874 | Val Loss: 0.4810 | Val
 Acc: 0.8647
 Epoch 9/40 | Train Loss: 0.3763 | Train Acc: 0.9010 | Val Loss: 0.4587 | Val
 Acc: 0.8682
 Epoch 10/40 | Train Loss: 0.3398 | Train Acc: 0.9117 | Val Loss: 0.4102 | Val
 Acc: 0.8830
 Epoch 11/40 | Train Loss: 0.3112 | Train Acc: 0.9182 | Val Loss: 0.4049 | Val
 Acc: 0.8785
 Epoch 12/40 | Train Loss: 0.2873 | Train Acc: 0.9227 | Val Loss: 0.3923 | Val
 Acc: 0.8808
 Epoch 13/40 | Train Loss: 0.2685 | Train Acc: 0.9285 | Val Loss: 0.3744 | Val
 Acc: 0.8878
 Epoch 14/40 | Train Loss: 0.2514 | Train Acc: 0.9317 | Val Loss: 0.3339 | Val
 Acc: 0.9033
 Epoch 15/40 | Train Loss: 0.2379 | Train Acc: 0.9340 | Val Loss: 0.3407 | Val
 Acc: 0.8993
 Epoch 16/40 | Train Loss: 0.2260 | Train Acc: 0.9364 | Val Loss: 0.3293 | Val
 Acc: 0.8978
 Epoch 17/40 | Train Loss: 0.2157 | Train Acc: 0.9390 | Val Loss: 0.3102 | Val
 Acc: 0.9097
 Epoch 18/40 | Train Loss: 0.2065 | Train Acc: 0.9403 | Val Loss: 0.3120 | Val
 Acc: 0.9083
 Epoch 19/40 | Train Loss: 0.1996 | Train Acc: 0.9419 | Val Loss: 0.3124 | Val
 Acc: 0.9073
 Epoch 20/40 | Train Loss: 0.1928 | Train Acc: 0.9440 | Val Loss: 0.2991 | Val
 Acc: 0.9128
 Epoch 21/40 | Train Loss: 0.1865 | Train Acc: 0.9453 | Val Loss: 0.3001 | Val
 Acc: 0.9107
 Epoch 22/40 | Train Loss: 0.1819 | Train Acc: 0.9446 | Val Loss: 0.2893 | Val
 Acc: 0.9137
 Epoch 23/40 | Train Loss: 0.1770 | Train Acc: 0.9470 | Val Loss: 0.2844 | Val
 Acc: 0.9137
 Epoch 24/40 | Train Loss: 0.1729 | Train Acc: 0.9483 | Val Loss: 0.2766 | Val
 Acc: 0.9180
 Epoch 25/40 | Train Loss: 0.1681 | Train Acc: 0.9483 | Val Loss: 0.2892 | Val
 Acc: 0.9110
 Epoch 26/40 | Train Loss: 0.1655 | Train Acc: 0.9493 | Val Loss: 0.3070 | Val
 Acc: 0.9087
 Epoch 27/40 | Train Loss: 0.1622 | Train Acc: 0.9507 | Val Loss: 0.2779 | Val
 Acc: 0.9177
 Epoch 28/40 | Train Loss: 0.1594 | Train Acc: 0.9506 | Val Loss: 0.2769 | Val
 Acc: 0.9182

```

Epoch 29/40 | Train Loss: 0.1574 | Train Acc: 0.9515 | Val Loss: 0.2728 | Val
Acc: 0.9190
Epoch 30/40 | Train Loss: 0.1550 | Train Acc: 0.9515 | Val Loss: 0.2760 | Val
Acc: 0.9190
Epoch 31/40 | Train Loss: 0.1515 | Train Acc: 0.9528 | Val Loss: 0.2779 | Val
Acc: 0.9177
Epoch 32/40 | Train Loss: 0.1497 | Train Acc: 0.9530 | Val Loss: 0.2667 | Val
Acc: 0.9167
Epoch 33/40 | Train Loss: 0.1480 | Train Acc: 0.9541 | Val Loss: 0.2744 | Val
Acc: 0.9183
Epoch 34/40 | Train Loss: 0.1454 | Train Acc: 0.9545 | Val Loss: 0.2859 | Val
Acc: 0.9133
Epoch 35/40 | Train Loss: 0.1438 | Train Acc: 0.9554 | Val Loss: 0.2657 | Val
Acc: 0.9198
Epoch 36/40 | Train Loss: 0.1429 | Train Acc: 0.9543 | Val Loss: 0.2871 | Val
Acc: 0.9142
Epoch 37/40 | Train Loss: 0.1408 | Train Acc: 0.9542 | Val Loss: 0.2689 | Val
Acc: 0.9188
Epoch 38/40 | Train Loss: 0.1399 | Train Acc: 0.9551 | Val Loss: 0.2695 | Val
Acc: 0.9177
Epoch 39/40 | Train Loss: 0.1390 | Train Acc: 0.9558 | Val Loss: 0.2612 | Val
Acc: 0.9200
Epoch 40/40 | Train Loss: 0.1383 | Train Acc: 0.9555 | Val Loss: 0.2772 | Val
Acc: 0.9162

```

```

[5]: save_dir = Path(PROJECT_DIR) / "models"
      save_dir.mkdir(parents=True, exist_ok=True)

      torch.save(net.state_dict(), save_dir / "model.pt")

      history = {
          "total_loss_train": total_loss_train,
          "total_accuracy_train": total_accuracy_train,
          "total_loss_valid": total_loss_valid,
          "total_accuracy_valid": total_accuracy_valid,
          "all_confusion_matrices": all_confusion_matrices,
          "input_dim": X_train.shape[1],
      }

      with open(save_dir / "history.pkl", "wb") as f:
          pickle.dump(history, f)

      print("model and history saved")

```

model and history saved